数据库系统第二章笔记：关系模型与代数

第一部分：关系模型基础

关系模式和实例

- $A1$，$A2$，…，An 是 *属性*
- $R = (A1, A2, \ldots, An)$ 是 *关系模式*

例子：

```
instructor = (ID, name, dept_name, salary)
```

- 关系实例 r 在模式 R 上定义 用 $r(R)$ 表示。
- 关系的当前值由表指定
- 元素 t 的 关系 r 被称为 *元组* 并由 *一行表示* 在表格中

1. 定规则（关系模式）

- **定义**：就是那张只有表头的空表格。它规定了这张表叫什么、有几列、每列叫什么（属性）。
- **讲师实例**：讲师 = (ID， 姓名， 部门， 薪水)
- **一句话**：它是“模板”，只有结构，没有数据。

2. 填格子（属性值）

- **定义**：往表头下面的格子里填的具体内容。
- **讲师实例**：ID 格子里填的“101”，姓名格子里填的“张三”。
- **一句话**：它是“格子里的内容”，是对属性的具体赋值。

3. 成一行（元组）

- **定义**：由同一行的所有格子（属性值） 组成的一条完整记录。
- **讲师实例**：`（101， 张三， 计算机系， 8000）`这一整行。
- **一句话**：它是“表格里的一行”，代表一个具体的人或事物。

4. 得全表（关系实例）

- **定义**：由所有行（元组）拼成的填满数据的完整表格。
- **讲师实例**：包含张三、李四、王五等所有人的那张大表。
- **一句话**：它是“整张表的数据”，是某一时刻的数据快照。

5. 完整关系

- **定义**：关系模式 + 关系实例 = 数据库里实际存在的表。
- **讲师实例**：讲师表的模板（ID / 姓名 / 部门 / 薪水）+ 所有讲师的实际数据 = 数据库中真实的讲师表。
- **一句话**：它是“模板 + 数据”的结合体，是数据库中实际操作的对象。

属性

- 允许值集合称为域 属性
- 属性值（通常）要求是原子的；也就是说，不可分割
- 特殊价值 *null*是每个域的成员。表示该值“未知”
- 空值导致许多操作的定义变得复杂

6. 属性的约束（域）

- **定义**：每个属性允许取值的集合，决定了格子能填什么内容。
- **讲师实例**：“薪水”属性的域是**“正数”，**不能填负数或文本；“ID”的域是**“唯一标识字符 / 数字”**，不能重复。
- **一句话**：它是“属性的取值规则”，规定了格子里能填什么、不能填什么。

7. 属性的原子性

- **定义**：属性值必须是**不可分割**的最小单元。
- **讲师实例**：“姓名”格里只能填一个名字，不能把“张三、李四”填在一起；“部门”格里不能填“计算机系 + 数学系”。
- **一句话**：它是“属性值的格式要求”，一个格子只能存一个不可拆的内容。

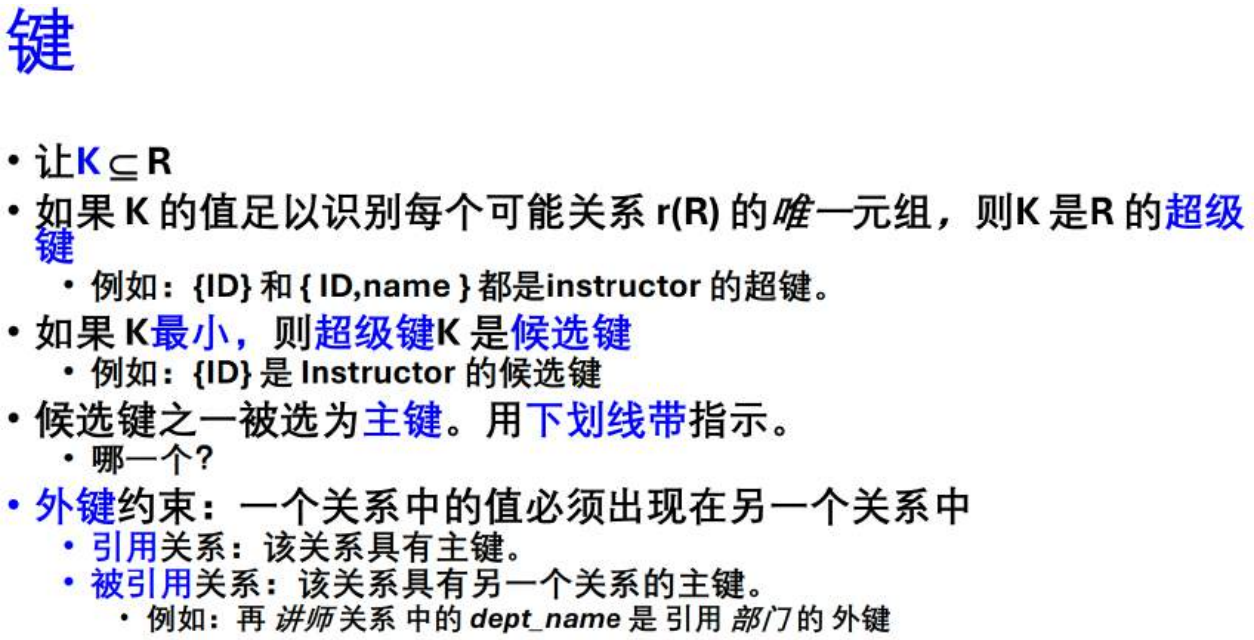
8. 特殊值 null

- **定义**：表示“值未知”或“不存在”，是所有属性域的默认成员。
- **讲师实例**：新讲师的“薪水”未定，这一格就填**null**；讲师的部门暂未分配，部门格也填**null**。
- **一句话**：它是“属性的空值标识”，专门用来表示未知 / 未填写的内容。

9. 关系的无序性

- **定义**：关系中的元组（行）没有固定的存储 / 排列顺序，元组顺序调换，关系实例的本质不变（因关系是元组的集合，集合元素无顺序）。

- **讲师实例**：把讲师表中`(101,张三,计算机系,8000)`和`(102,李四,数学系,7500)`的行互换，依然是同一个讲师表。
- **一句话**：它是“元组的排列规则”，表格里的行谁在前、谁在后，不影响数据本身的含义。

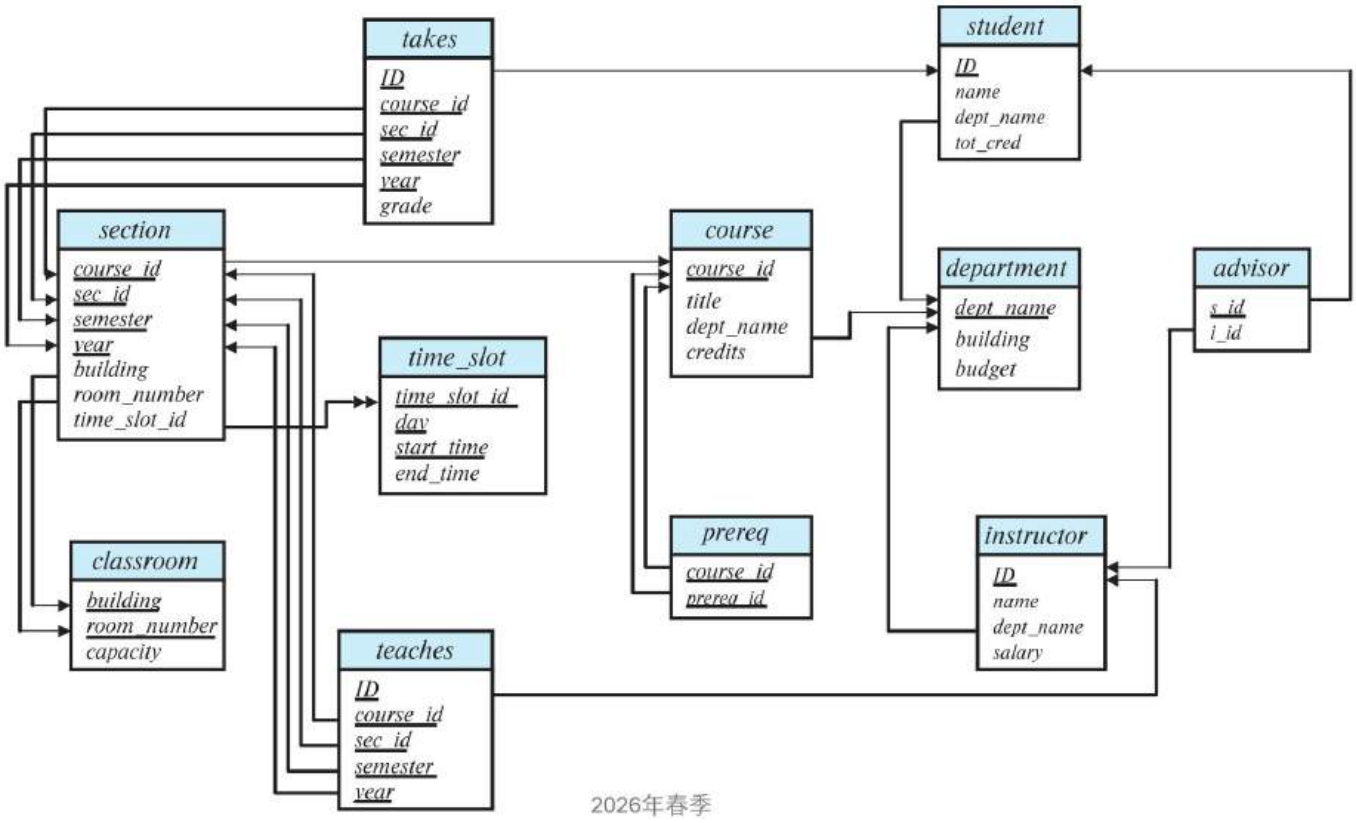


10. 键的核心概念（超键 / 候选键 / 主键 / 外键）

- **超键**：能唯一识别表中每个元组的**一个 / 一组属性**，可以包含多余属性。
  - 讲师实例：{ID}、{ID, 姓名} 都能唯一识别讲师，都是讲师表的超键。
  - 一句话：“能唯一找出行的属性 / 属性组”，可以多带无关属性。
- **候选键**：**最小的超键**，去掉任何一个属性就无法唯一识别元组。
  - 讲师实例：{ID} 是讲师表的候选键（去掉 ID 就无法唯一识别讲师）。
  - 一句话：“最简的超键”，不多带一个多余属性。
- **主键**：从候选键中**选定的一个**，作为表的唯一标识，通常用下划线标注。
  - 讲师实例：选定 {ID} 作为讲师表的主键，是识别讲师的官方唯一标识。
  - 一句话：“官方选定的候选键”，表的核心唯一标识。
- **外键**：一个表中的属性 / 属性组，其值必须**匹配另一个表的主键**，用于关联两个表。
  - 讲师实例：讲师表的“部门”属性，是引用**部门表**主键的外键，讲师的部门必须是部门表中存在的部门。
  - 一句话：“表之间的关联纽带”，让两个表能通过这个属性建立联系。

11. 易混概念区分

- **属性 vs 属性值**：“姓名”（属性）是表头栏目名；“张三”（属性值）是栏目里的具体内容。
- **模式 vs 实例**：“空表”（模式）是数据蓝图 / 模板；“填满的表”（实例）是模板填完数据的结果。
- **元组 vs 实例**：“一行数据”（元组）是单个个体记录；“整张表数据”（实例）是所有个体的记录集合。
- **候选键 vs 主键**：候选键是“多个可选的最简唯一标识”；主键是“从候选键中挑出的一个官方标识”。



第二部分：键与关系类型

1. 键的识别与定义

- **考题示例**：
  1. 指出 `section` 表的主键是什么？为什么需要联合主键？
  2. `takes` 表中的 `ID` 字段是什么键？它引用了哪个表的哪个字段？
- **解题思路**：
  - **主键**：看**下划线**，`section` 的主键是 `(course_id, sec_id, semester, year)`，因为单字段无法唯一标识一次开课。
  - **外键**：看**连线**，`takes.ID` 是外键，引用 `student.ID`，保证选课学生必须存在。

2. 表的作用与关系类型

- **考题示例**：
  1. `teaches` 表的作用是什么？它连接了哪两个实体？
  2. 学生和课程之间是多对多关系吗？为什么需要 `takes` 表？
- **解题思路**：
  - `teaches` 表记录“讲师授课”关系，连接 `instructor` 和 `section`。
  - 学生和课程是多对多（一个学生选多门课，一门课被多个学生选），`takes` 表用来分解多对多关系，避免数据冗余。

关系查询语言

Procedural Non-procedural

- 过程式 与 非过程式或声明式
- “纯”语言：
  - 关系代数
  - 元组关系演算
  - 领域关系演算
- 上述 3 种纯语言在计算能力上是相当的
- 本次课 我们将集中讨论 关系代数
  - 非图灵机等价物
  - 6个基本操作组成

- **过程式**：告诉数据库「怎么做」才能拿到数据（比如：先筛行→再选列→最后合并），关系代数就是这类。
- **非过程式 / 声明式**：只告诉数据库「要什么」，不用管“怎么做”（比如：直接说“要计算机系学生姓名”），SQL 偏这类。
- **“纯”语言**：数据库理论层面的“基础语言”，关系代数、元组 / 域关系演算这 3 种能力完全一样，只是表达形式不同。

第三部分：关系代数

关系代数

procedural language

- 一种过程语言，由一组操作组成，这些操作以一个或两个关系作为输入，并产生一个新的关系作为结果。
- 六个基本运算符
  - 选择：σ（sigma）
  - 项目：π（Pi）
  - 并集：∪（交集：∩）
  - 集合差异：－
  - 笛卡尔积：x
  - 重命名：ρ（Rho）

- **核心逻辑**：就像给数据库下达一步步的操作指令。
- **通俗理解**：“先做这个，再做那个，然后把结果给我。”
- **表达示例**：
  - **第一步**：`σ_dept_name="Comp.Sci."(instructor)` —— 筛选出所有**计算机系的行**。
  - **第二步**：`π_name(...)` —— 从上一步的结果中，只切出**姓名列**。

- **一句话**：你清晰地告诉数据库“操作的步骤”**。

操作名称	符号	通俗动作	新手例子（结合大学数据库）
选择	σ	筛行（挑符合条件的行）	σ_dept_name='Comp. Sci.'(student) → 只留「计算机系学生」的行（其他系的行都删掉）
投影	π	选列（只留想要的列）	π_name,tot_cred (student) → 只保留「姓名、总学分」两列（ID、院系列都删掉）
并集	∪	合并行（去重）	（2023 春选课的学生 ID）∪（2023 秋选课的学生 ID）→ 所有选过课的学生 ID（重复的只留一个）
集合差异	−	找 “独有的行”	（所有学生 ID）−（选课的学生 ID）→ 没选过任何课的学生 ID
笛卡尔积	×	行两两组合	student × course → 每个学生和每门课都组合一次（比如张三 + CS-101、张三 + Math-201…，结果会非常大）
重命名	ρ	给表 / 列改名	ρ_stu (student) → 把 student 表改名叫 stu（复杂查询里避免表名重复）

1. 选择运算 (Selection)

1. 核心本质

选择操作（σ）是关系代数里最基础的**“筛行工具”** —— 它只做一件事：从指定的表（关系）里，挑出**满足条件的行（元组）**，不删列、不新增列，输出的新表和原表列数一样，只是行变少了（只留符合条件的）。

2. 符号拆解：$\sigma_p(r)$

这个符号是选择操作的 “固定写法”，每部分都有明确含义：

符号 / 部分	通俗含义
σ	选择操作的专属标识，记住 “σ= 筛行” 就行
p	选择谓词 = 筛选条件（比如 “部门 = 物理”“薪水 > 90000”），是筛选的 “规则”
r	要筛选的表（关系），比如 `instructor`（讲师表）、`department`（部门表）

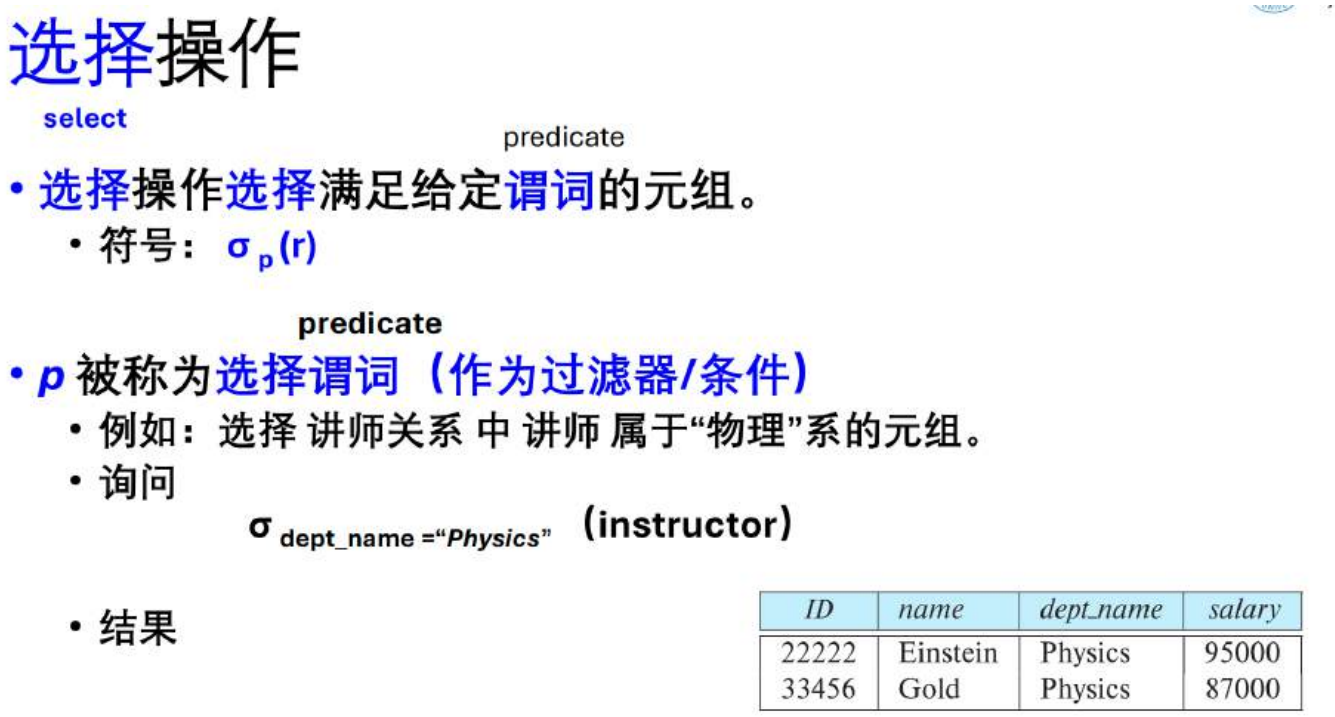
3. 筛选条件写法

这段内容里明确了条件的两种写法，新手记这两类就够：

1. **基础条件**：用比较符写单个规则
    - `=`：等于（比如 `dept_name="Physics"` → 部门是物理系）
    - `≠`：不等于（比如 `dept_name≠"Math"` → 部门不是数学系）
    - `>`：大于（比如 `salary>90000` → 薪水高于 9 万）
    - `≥`：大于等于（比如 `salary≥80000` → 薪水≥8 万）
    - `<`：小于、`≤`：小于等于（用法同上）

2. **组合条件**：用连接词把多个规则拼起来
    - ∧：和（且）→ 多个条件**必须同时满足**
    - ∨：或 → 多个条件**满足一个就行**
    - ¬：否 → 取“条件的反面”
3. **特殊条件**：表内两个属性之间比较
    - 条件可以不是“属性和固定值比”，而是“表中两个属性互比”（比如“部门名 = 建筑物名”）。

4. 实例解析



结合内容里的 3 个例子，翻译成大白话：

1. **筛选物理系的讲师**
    - **公式**：$\sigma_{\text{dept_name="Physics"}}(\text{instructor})$
    - **解读**：从「讲师表（instructor）」里，只挑出“dept_name（所属部门）等于 Physics（物理系）”的行（所有物理系讲师的完整记录，列不变）。
2. **筛选薪水超 9 万的物理系讲师**
    - **公式**：$\sigma_{\text{dept_name="Physics"} \wedge \text{salary>90000}}(\text{instructor})$
    - **解读**：从讲师表中，挑出“既是物理系（dept_name=Physics），又满足薪水> 90000 美元”的行（两个条件必须同时满足）。
    - ✅ **拓展**：如果想查“物理系 或 薪水超 9 万”的讲师，把∧换成∨即可。
3. **筛选“部门名 = 建筑物名”的部门**
    - **公式**：$\sigma_{\text{dept_name=building}}(\text{department})$
    - **解读**：从「部门表（department）」里，挑出“部门名称（dept_name）和所在建筑物名称（building）相同”的行（比如“数学系”刚好在“数学系楼”，就会被筛出来）。

选择操作（续）

- 我们允许使用

  $=、\neq、>、\geq、<、\leq$

  在选择谓词中。
- 我们可以使用连接词将几个谓词组合成一个更大的谓词：

  $\wedge$（和），$\vee$（或），$\neg$（否）
- 例如：查找薪水高于 90,000 美元的物理学讲师，我们写为：

$$\sigma_{dept_name=\text{“Physics”} \wedge salary > 90{,}000}(instructor)$$

- 选择谓词可能包括两个属性之间的比较。
  - 例如，查找所有名称与建筑物名称相同的部门：

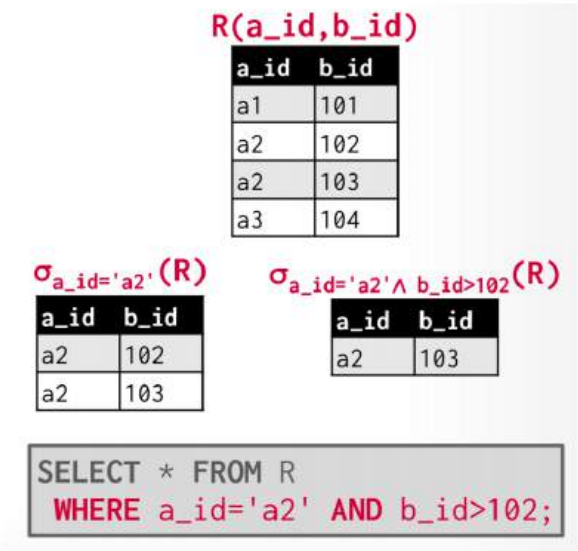
$$\sigma_{dept_name=building}(department)$$

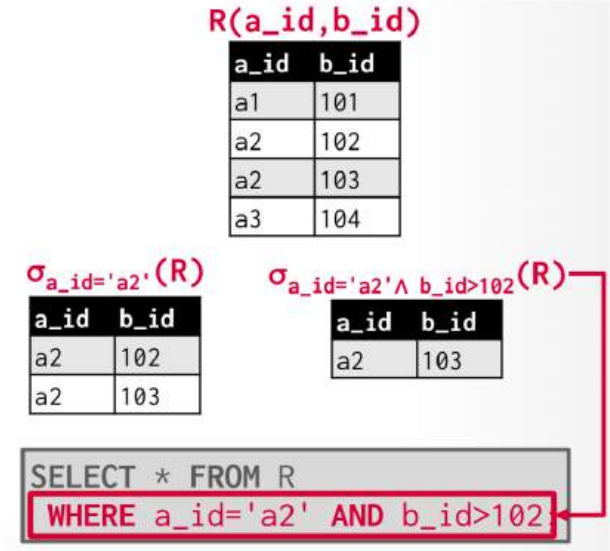
5. 总结

1. 选择操作（σ）核心是**“筛行不筛列”**：只保留满足条件的行，列数和原表一致；
2. 公式写法：σ+「筛选条件」+（表名），条件可以是单条件、多条件组合，甚至属性互比；
3. 新手关键：先练“单条件筛选”（比如筛物理系讲师），再练“多条件组合”（比如物理系 + 高薪），就能掌握选择操作的核心。
4. 简单记：选择操作 = 给表“按条件挑行”，σ 是挑行的标识，条件写在 σ 后面，表名写在括号里。

练习：

选择操作示例





第一步

- **关系代数写法**：σa_id='a2'(R)
- **大白话解读**：从表 R 里，只挑出 a_id 等于 'a2' 的那些行。

```
-- 查询所有列（*）从指定的表里
SELECT *          -- * 是通配符，代表表中的所有列（属性）
FROM R            -- R 是表名（关系实例），指定要从哪个表查数据
WHERE a_id='a2'     -- 筛选条件1：只保留 a_id 列的值等于 'a2' 的行
```

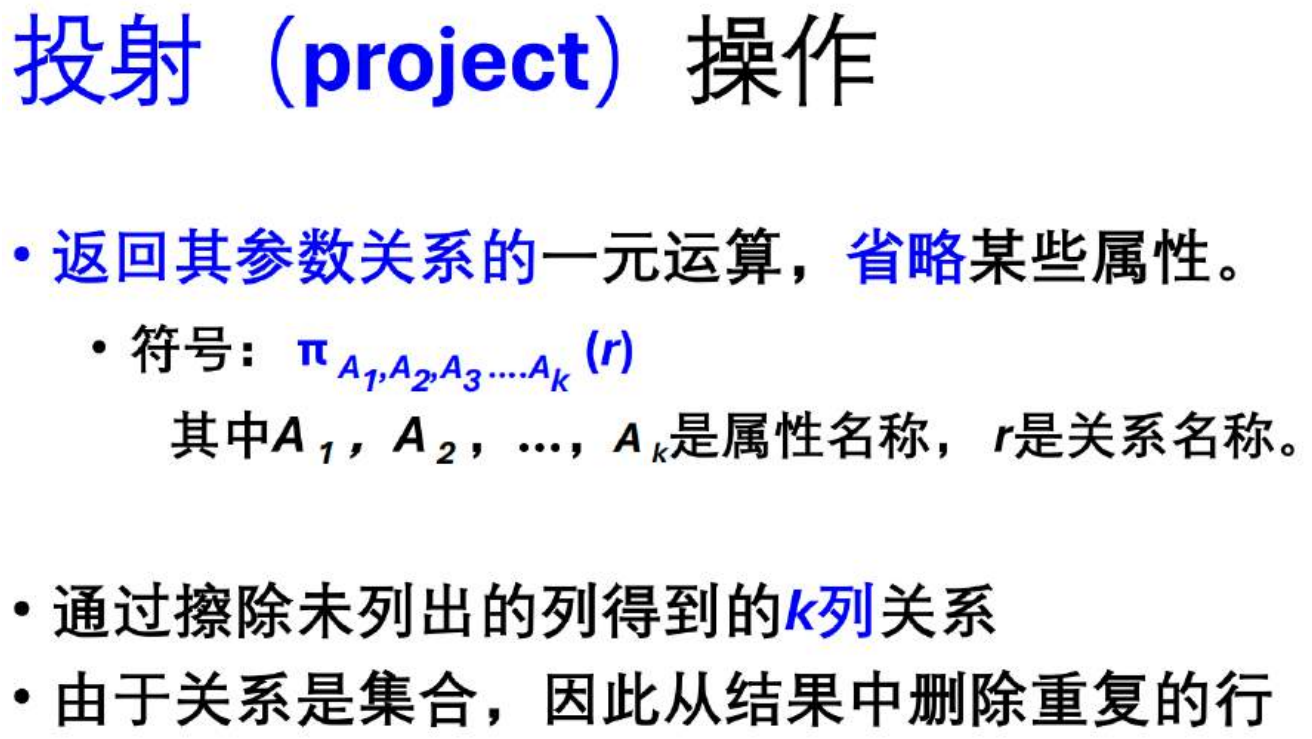
第二步，增加条件

- **关系代数写法**：σa_id='a2'∧b_id>102(R)
- **大白话解读**：从表 R 里，挑出同时满足两个条件的行：
  - a_id 等于 'a2'
  - b_id 大于 102

```
-- 查询所有列（*）从指定的表里
SELECT *          -- * 是通配符，代表表中的所有列（属性）
FROM R            -- R 是表名（关系实例），指定要从哪个表查数据
WHERE a_id='a2'     -- 筛选条件1：只保留 a_id 列的值等于 'a2' 的行
  AND b_id>102;     -- 筛选条件2：同时要求 b_id 列的值大于 102
```

2. 投影运算 (Projection)

1. 核心本质



投影操作（π）是关系代数里的**“选列工具”**，和之前讲的选择操作（σ，筛行）刚好对应：

- **选择（σ）**：只筛行、不碰列（保留所有列，只删不符合条件的行）；
- **投影（π）**：只选列、可去重（保留指定列，删掉没列出的列，结果自动删重复行）。

选中内容里明确说它是“一元运算”—— 意思是**只需要一个表（关系）就能操作**，不用像合并表那样需要两个表。

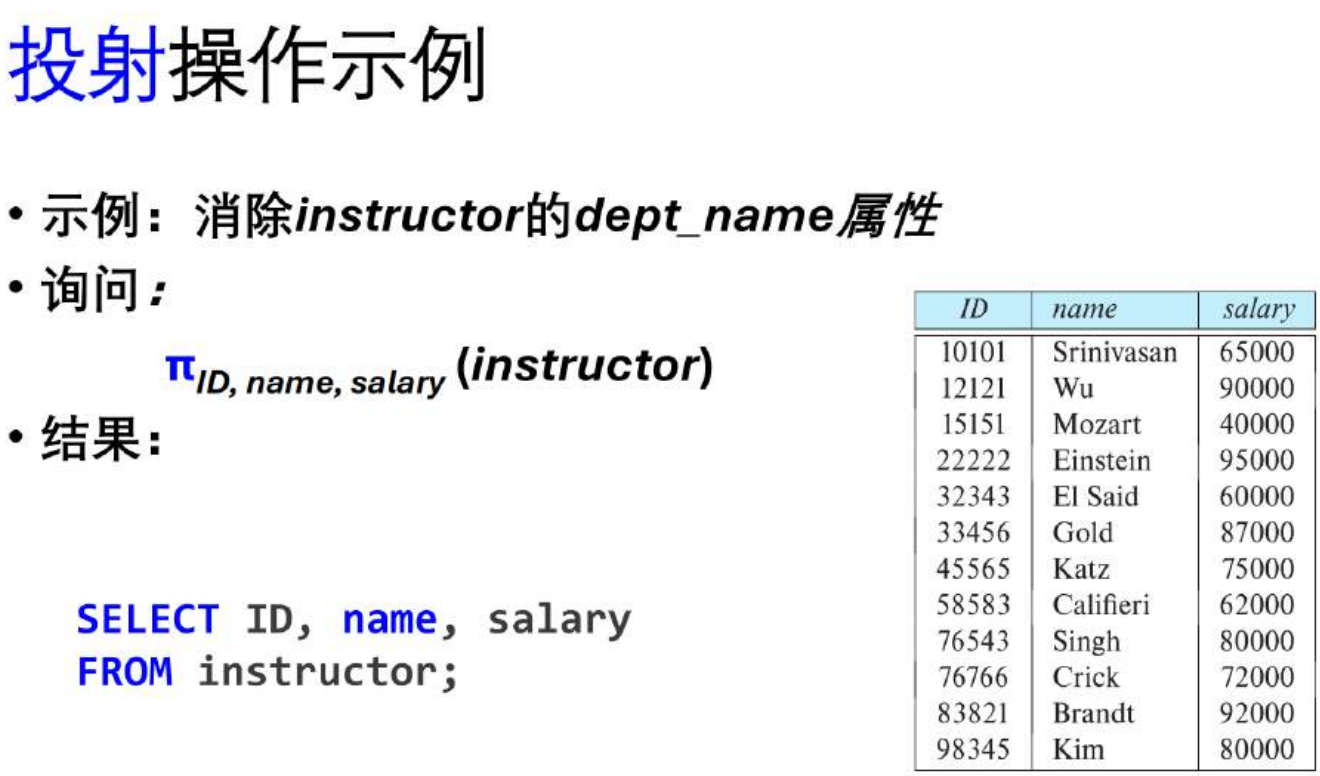
2. 符号拆解：π A1,A2...Ak (r)

这个符号是投影的固定写法，每部分都有明确含义：

符号 / 部分	通俗含义
π	投影操作的专属标识，新手记“π= 选列”就行
A1,A2... Ak	你想保留的属性（列名），比如 ID、name、salary（没列出来的列会被“擦掉”）
r	要操作的表（关系），比如选中内容里的`instructor`（讲师表）

核心规则：投影的结果是一个“k 列关系”——k 就是你选的列数，比如选 ID、name、salary 这 3 列，结果就只有 3 列。

3. 示例解读



选中内容里的例子是“消除 instructor 的 dept_name 属性”，我们拆成两步理解：

1. **关系代数写法**

```
π ID, name, salary (instructor)
```

- **解读**：对`instructor`（讲师表）做投影操作，只保留`ID`（讲师编号）、`name`（姓名）、`salary`（薪水）这 3 列，**删掉`dept_name`（所属部门）列**；
- **关键**：因为关系是“集合”，如果有两行的 ID、name、salary 完全一样（比如两个讲师 ID / 姓名 / 薪水都相同），结果会自动删掉重复行。

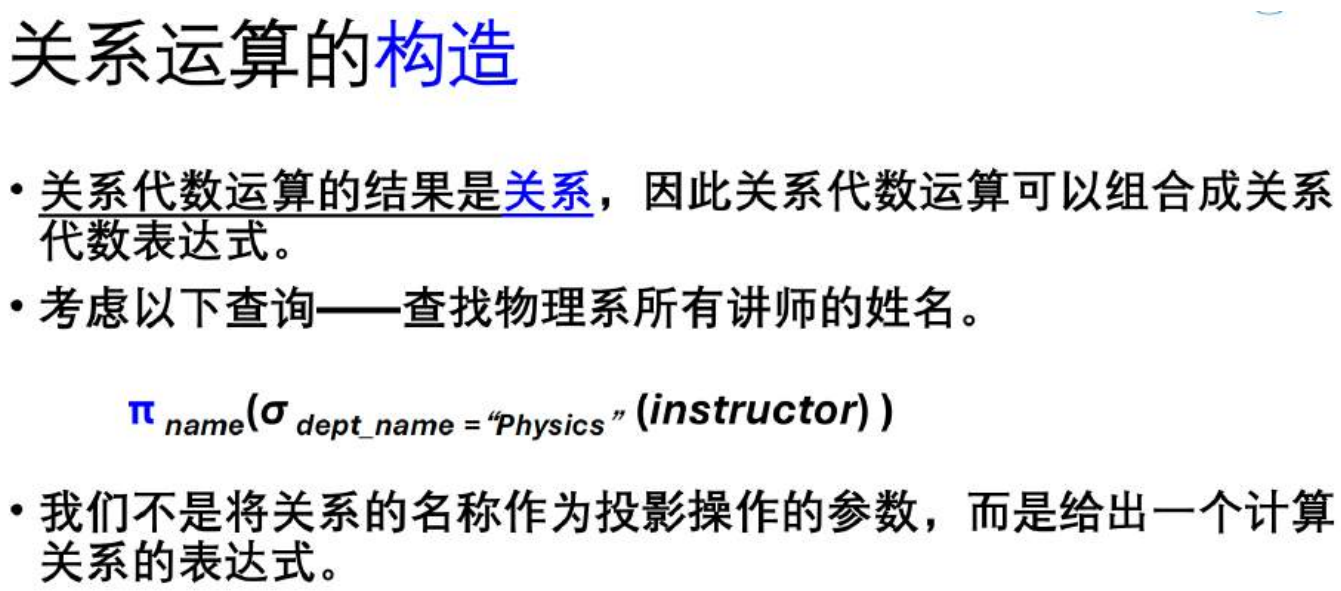
2. **对应的 SQL 写法**

```
-- 查询指定列：只从表里取出需要的字段，不查所有列
SELECT
    ID,         -- 讲师编号：唯一标识每一位讲师
    name,       -- 讲师姓名：讲师的姓名
    salary      -- 讲师薪水：讲师的工资数额
FROM
    instructor; -- 讲师表：数据来源的关系实例
```

- **新手对应**：SQL 的`SELECT 列名`就是投影操作的“选列”逻辑，`FROM 表名`对应投影里的`r`（操作的表）；
- **补充**：SQL 默认不会自动去重，如果想和投影一样去重，需要加`DISTINCT`（比如`SELECT DISTINCT ID, name, salary FROM instructor;`）。

4. 组合运算

投影操作的参数不只是“表名”，还能是“另一个关系代数运算的结果”—— 这是关系代数能实现复杂查询的核心，选中内容里的例子“查找物理系所有讲师的姓名”就是典型：



1. **需求拆解** 要查“物理系所有讲师的姓名”，需要两步：
   - ① 先筛行：从讲师表里挑出`dept_name="Physics"`（物理系）的行（用选择操作 σ）；
   - ② 再选列：只保留这些行的`name`（姓名）列（用投影操作 π）。
2. **组合后的关系代数表达式**

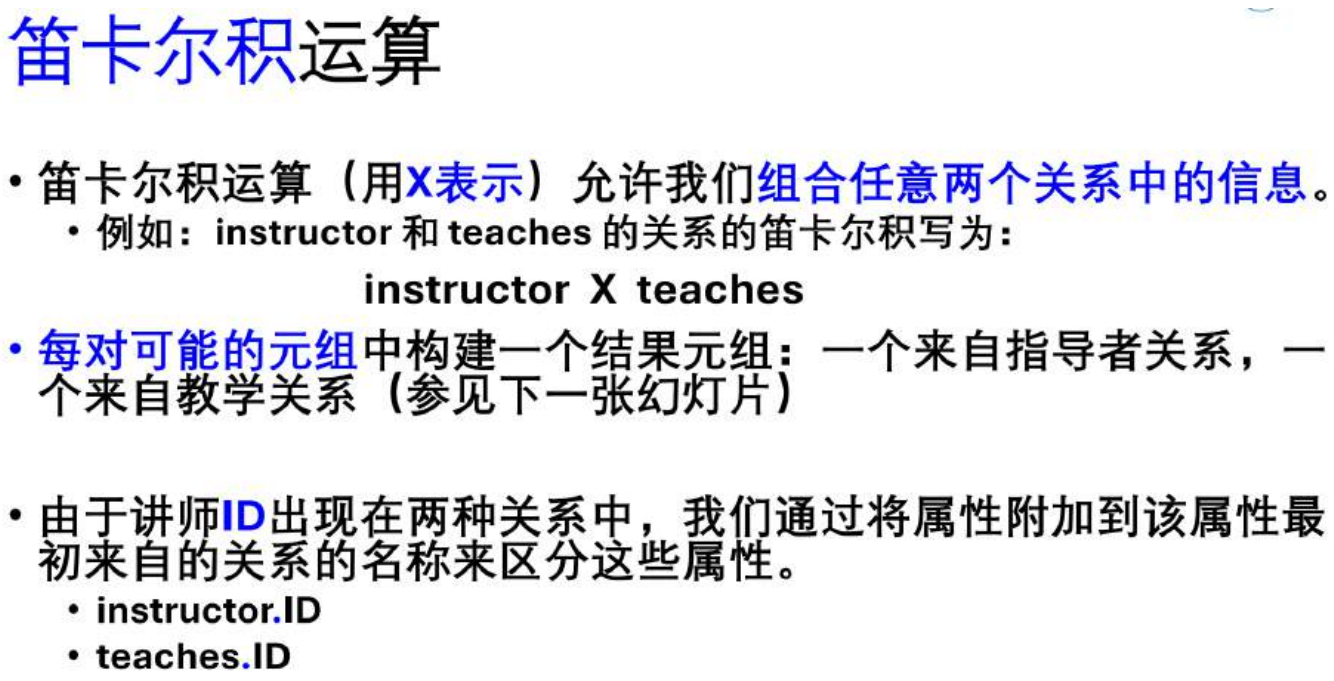
```
π name ( σ dept_name="Physics" (instructor) )
```

- **解读**：括号里的`σ dept_name="Physics" (instructor)`是第一步（筛出物理系讲师的完整行），把这个“临时结果表”作为投影操作的参数，再只选`name`列；
- **逻辑顺序**：**先做括号里的操作（筛行），再做括号外的操作（选列）**，最终结果只有“物理系讲师的姓名”这一列，没有其他冗余信息。

5. 总结

1. 投影（π）核心是**选列 + 去重**：只保留指定列，删掉未列出的列，结果自动删除重复行；
2. 符号写法：π + 要保留的列名 + (操作的表 / 其他运算结果)；
3. 组合运算：投影可以和选择（σ）嵌套使用，先筛行、再选列，实现精准查询（比如只查物理系讲师的姓名）；
4. 和 SQL 对应：投影的“选列”对应 SQL 的`SELECT 列名`，去重需加`DISTINCT`。

3. 笛卡尔积 (Cartesian Product)



1. 核心本质

笛卡尔积（×）是关系代数里的**“行两两组合工具”**，和选择 / 投影的核心区别很明显：

- **选择（σ）、投影（π）**：**一元运算**（只需要 1 个表就能操作），要么筛行、要么选列；
- **笛卡尔积（×）**：**二元运算**（必须有 2 个表），把表 A 的**每一行**和表 B 的**每一行**都“配对”，生成新表。

> 表 A 有 10 行，表 B 有 5 行，笛卡尔积的结果就有 10×5=50 行；表 A 有 3 列，表 B 有 4 列，结果就有 3+4=7 列。

2. 符号与写法

- **专属标识**：`×`（选中内容写的`X`是同一个意思，只是书写形式不同）；

- **基本写法**：`表1 × 表2`，比如选中内容里的 `instructor × teaches`（讲师表 × 授课表）。

3. 属性重名处理

这是笛卡尔积最关键的细节，也是选中内容重点强调的点：

- **问题**：`instructor`（讲师表）和`teaches`（授课表）都有`ID`列（讲师 ID），如果直接组合，会出现两个“ID”，无法区分；
- **解决方法**：给重名的属性**加“表名。属性名”的前缀**，比如：
  - 讲师表的 ID → `instructor.ID`；
  - 授课表的 ID → `teaches.ID`；
- **作用**：明确每个属性来自哪个表，避免混淆。

4. 实例解析

假设：

- `instructor`表（2 行）：

instructor.ID	name	dept_name	salary
101	张三	计算机系	8000
102	李四	数学系	7500

- `teaches`表（2 行）：

teaches.ID	course_id	sec_id	semester	year
101	CS-101	1	Spring	2026
102	Math-201	1	Spring	2026

执行 `instructor × teaches` 后，结果有 2×2=4 行，列数 = 4+4=8 列（重名的 ID 已加前缀）：

instructor.ID	name	dept_name	salary	teaches.ID	course_id	sec_id	semester	year
101	张三	计算机系	8000	101	CS-101	1	Spring	2026
101	张三	计算机系	8000	102	Math-201	1	Spring	2026
102	李四	数学系	7500	101	CS-101	1	Spring	2026
102	李四	数学系	7500	102	Math-201	1	Spring	2026

👉 **关键观察**：结果里有“无效配对”（比如张三配了李四的授课记录），这也是笛卡尔积很少单独用的原因。

5. 实际用途

笛卡尔积几乎不会单独用，而是**和选择操作（σ）结合**，筛选出“有效配对”（比如只保留`instructor.ID = teaches.ID`的行），这就是“连接运算”的核心逻辑。

比如想查“每个讲师实际授课的课程”，需要：

1. 先做笛卡尔积：`instructor × teaches`；
2. 再用选择筛有效行：`σ instructor.ID = teaches.ID (instructor × teaches)`；
3. 最后用投影选需要的列：`π instructor.ID, name, course_id (σ instructor.ID = teaches.ID (instructor × teaches))`。

最终结果会删掉无效配对，只保留“讲师 ID 和授课 ID 一致”的行，得到：

instructor.ID	name	course_id
101	张三	CS-101
102	李四	Math-201

6. 总结

1. 笛卡尔积（×）是**二元运算**，核心是“两个表的行两两配对”，结果行数 = 表 1 行数 × 表 2 行数，列数 = 表 1 列数 + 表 2 列数；
2. 重名属性需加`表名.属性名`前缀（比如`instructor.ID`），避免混淆；
3. 笛卡尔积很少单独用，通常和选择（σ）结合筛选有效行，再用投影（π）选列，实现“表连接查询”。
4. 简单记：笛卡尔积 =“全量配对”，选择 =“筛有效配对”，投影 =“选需要的列”，三者组合就能实现多表关联查询。

4. 连接运算 (Join)

1. 为什么需要连接操作？

之前我们讲过笛卡尔积（×）的问题：

- 执行 `instructor × teaches`（讲师表 × 授课表）时，会把**每个讲师的行**和**每个授课记录的行**都强行配对；
- 结果里大部分行都是“无效垃圾数据”（讲师和非自己的授课记录配对）。

所以之前我们要做两步：

1. 先做笛卡尔积：`instructor × teaches`（全量配对，出很多无效行）；
2. 再用选择操作筛有效行：`σ instructor.id = teaches.id (instructor × teaches)`（只留讲师 ID 和授课 ID 一致的行）。

2. 连接操作的核心

选中内容里明确说：**连接操作允许我们将选择操作和笛卡尔积运算合并为单一运算。**

1. **连接操作的通用公式**

```
r ⋈θ s = σθ (r × s)
```

符号 / 部分	通俗含义
⋈θ	连接操作的标识，θ 是“筛选条件”（比如 instructor.id=teaches.id）
r × s	先做 r 和 s 的笛卡尔积（全量配对）
σθ	再按 θ 条件筛选有效行

👉 **大白话**：连接操作就是“先把两个表的行全量配对，再直接筛出符合条件的行”，不用分两步写，一步到位。

2. **对应到讲师 / 授课表的例子**
   - 之前要写两行的操作：

     ```
     σ instructor.id = teaches.id (instructor × teaches)
     ```

   - 用连接操作可以**简写为一行**：

     ```
     instructor ⋈instructor.id = teaches.id teaches
     ```

   - 这两个表达式的结果**完全一样**，只是连接操作更简洁，也更符合我们“查讲师自己授课记录”的直观逻辑。

3. 实例解析

假设：

- `instructor`表（2 行）：

id	name	dept_name
101	张三	计算机系
102	李四	数学系

- `teaches`表（2 行）：

id	course_id
101	CS-101
102	Math-201

步骤 1：笛卡尔积 `instructor × teaches`（4 行，含无效行）

instructor.id	name	dept_name	teaches.id	course_id	
101	张三	计算机系	101	CS-101	（有效）
101	张三	计算机系	102	Math-201	（无效：张三配李四的课）
102	李四	数学系	101	CS-101	（无效：李四配张三的课）
102	李四	数学系	102	Math-201	（有效）

步骤 2：连接操作 `instructor ⋈instructor.id=teaches.id teaches`（直接出 2 行有效行）

instructor.id	name	dept_name	teaches.id	course_id
101	张三	计算机系	101	CS-101

instructor.id	name	dept_name	teaches.id	course_id
102	李四	数学系	102	Math-201

👉 **核心差异**：连接操作跳过了“生成无效行”的步骤，直接只保留有意义的配对，既简洁又高效。

4. 总结

1. 连接操作的本质：**笛卡尔积（全量配对）+ 选择操作（筛有效行）**，合并成一个操作，避免生成大量无效数据；
2. θ（读“theta”）就是筛选条件，新手最常见的 θ 是“两个表的关联字段相等”（比如 instructor.id=teaches.id），这种也叫「等值连接」；
3. 写法简化：不用写复杂的“σ+×”，直接用“⋈条件”就能实现多表的有效关联查询；
4. 实际用途：我们日常查“讲师教了什么课”“学生选了什么课”，本质都是用连接操作关联多个表。
5. 简单记：连接操作 = 有条件的笛卡尔积，条件就是 θ（比如 ID 相等）。

5. 联合运算 (Union)

1. 核心本质

联合运算（∪）是关系代数里的**“行合并工具”**，属于二元运算（必须有两个表 / 关系），核心作用是：**把两个表的所有行合并成一个新表，并且自动删掉重复的行**（比如同一门课 ID 在两个表都出现，只留一个）。

👉 **生活化比喻**：就像你有两张购物清单，一张是“2017 秋要买的课程 ID”，一张是“2018 春要买的课程 ID”，联合运算就是把这两张清单粘在一起，删掉重复的课程 ID，得到“所有要买的课程 ID 清单”。

2. 两个前提

联合运算不是随便两个表都能合并的，选中内容明确了两个“门槛”，少一个都不行：

1. **相同元数（相同列数）**
   - “元数”就是表的**列数**，比如：
   - 表 A 是 1 列（只有 course_id），表 B 也必须是 1 列；
   - 表 A 是 2 列（course_id、sec_id），表 B 也必须是 2 列；
   - ❌ **错误示例**：一个表 1 列，一个表 2 列 → 没法合并。
2. **属性域兼容（列的类型一致）**
   - “属性域”就是列的**数据类型**，比如：
   - 表 A 的唯一列是“课程 ID”（字符型，如 CS-101），表 B 的唯一列也必须是“课程 ID”（字符型）；
   - 不能表 A 的列是“数字”（如 101），表 B 的列是“文字”（如张三）→ 类型不兼容，没法合并。

3. 实例解析

选中内容的示例需求是：**查找 2017 年秋季、2018 年春季，或两个学期都教授的所有课程 ID**（只要课程 ID，不管重复）。

联合 操作

- 联合运算允许我们将两个关系结合起来：符号：$r \cup s$
- 对于 $r \cup s$ *是* 有效的，必须满足。
  1. r，s必须具有*相同*的元数（相同数量的属性）
  2. 属性域必须兼容（例如：第二个r列处理的值类型与s的第二列）
- 示例：查找 2017 年秋季学期、2018 年春季学期或两个学期教授的所有课程

$$\pi_{course_id}(\sigma_{semester=\text{“Fall”} \wedge year=2017}(section)) \cup \pi_{course_id}(\sigma_{semester=\text{“Spring”} \wedge year=2018}(section))$$

联合 操作（续）

- 结果：

$$\pi_{course_id}(\sigma_{semester=\text{“Fall”} \wedge year=2017}(section)) \cup \pi_{course_id}(\sigma_{semester=\text{“Spring”} \wedge year=2018}(section))$$

course_id
CS-101
CS-315
CS-319
CS-347
FIN-201
HIS-351
MU-199
PHY-101

1. **拆解左边的 “2017 秋课程 ID 清单”**

```
π course_id ( σ semester="Fall" ∧ year=2017 (section) )
```

   - 内层σ（选择操作）：从`section`（课段表）里，筛出 “学期 = Fall（秋季）且年份 = 2017” 的所有行；
   - 外层π（投影操作）：只保留这些行的`course_id`列 → 得到 “2017 秋开课的课程 ID 清单”（1 列）。

2. **拆解右边的 “2018 春课程 ID 清单”**

```
π course_id ( σ semester="Spring" ∧ year=2018 (section) )
```

- 内层σ：从section表里，筛出“学期 = Spring（春季）且年份 = 2018”的所有行；
- 外层π：只保留course_id列 → 得到“2018 春开课的课程 ID 清单”（1 列）。

3. **用∪合并两个清单**

```
[2017秋课程ID清单] ∪ [2018春课程ID清单]
```

- **结果**：把两个清单的课程 ID 合并，删掉重复的；
- **满足前提**：两个清单都是 1 列（相同元数），列都是 course_id（属性域兼容），所以联合运算有效。

4. 总结

1. 联合运算（∪）核心：**合并行 + 去重**，结果的列数和原表一致，行数≤两个表行数之和（因为删了重复）；
2. 两个硬前提：列数相同 + 每一列的类型一致（属性域兼容）；
3. 示例逻辑：先通过“选择（σ）筛行 + 投影（π）选列”得到两个符合条件的“单列清单”，再用∪合并去重；
4. 和 SQL 的对应：关系代数的∪对应 SQL 的UNION。